

RV32I Single-Cycle Processor

32-Bit Harvard-Architecture RISC-V Core — RTL Implementation

IEEE Cairo University Student Branch — Digital Design Final Project

Poula Kerolos

Electronics & Electrical Communications Engineering — Cairo University

July 2026

1. Introduction

This report documents the complete RTL design and functional verification of a 32-bit single-cycle RISC-V processor (RV32I base integer instruction set) implemented in Verilog and simulated in ModelSim. The processor was developed as the final project of the IEEE Cairo University Student Branch Digital Design Workshop and follows a Harvard microarchitecture — separate instruction and data memories — in which every instruction is fetched, decoded, executed, and written back within a single clock cycle.

The project required implementation of twelve hardware modules and a top-level interconnect, verified against a Fibonacci sequence program provided as machine code. All expected register-file and data-memory values were confirmed to match the specification's reference waveform.

2. Project Objectives

- Implement all submodules of a 32-bit single-cycle RISC-V processor in synthesisable RTL Verilog.
- Wire submodules into a correct top-level datapath matching the Harvard microarchitecture specified by IEEE CUSB.
- Verify the processor by executing a 21-instruction Fibonacci machine-code program and confirming all 10 output values stored in data memory.
- Produce a ModelSim automation script (.do file) that compiles, elaborates, and displays the correct simulation waveform.

3. Processor Architecture

3.1 Overview

The design is a Harvard-architecture single-cycle core: instruction memory and data memory are physically separate, and every stage (fetch, decode, execute, memory access, write-back) resolves within one clock period. There is no pipeline; a new instruction begins only when the previous one has fully completed and the PC has updated.

The supported instruction subset covers all instruction types used by the test program: R-type arithmetic (ADD, SUB, XOR, AND, OR, SLL, SRL), I-type arithmetic (ADDI, SLLI), load word (LW), store word (SW), and three branch types (BEQ, BNE, BLT).

3.2 Datapath Signal Flow

The execution path for a single cycle proceeds as follows:

- PC drives Instruction Memory, fetching a 32-bit instruction word.
- The opcode field is decoded by the Control Unit (Main Decoder + ALU Decoder + Branch Logic) to produce all control signals.
- rs1, rs2, and rd addresses are sliced directly from the instruction word and fed to the Register File, which outputs the two source operands immediately (asynchronous read).
- The Sign Extender reconstructs the immediate value from the appropriate bit fields depending on the instruction format (I, S, or B-type).
- A 2:1 MUX selects the ALU's second operand: the register value (RD2) for R-type instructions, or the sign-extended immediate for all other types.

- The ALU performs the operation specified by the 3-bit ALUControl signal and produces a result plus Zero and Sign flags.
- For store instructions, the ALU result is used as a memory address; the value to write comes from RD2.
- A second 2:1 MUX selects the write-back value: the ALU result for arithmetic instructions, or the memory read-data for load instructions.
- The selected value is written to the register file on the rising clock edge.
- Branch Logic examines the ALU flags and funct3 to decide PCSrc; the PC mux selects PC+4 (no branch) or PC+ImmExt (branch taken) for the next cycle.

4. Module Descriptions

4.1 Program Counter (program_counter.v)

The Program Counter is a 32-bit register that holds the byte address of the currently executing instruction. It advances to PCNext on every rising clock edge when load is asserted, and resets to zero when reset is deasserted (active-low asynchronous reset). The load signal is driven low by the HALT detection logic when an all-zero instruction is fetched, freezing the PC and ending execution.

```
module program_counter (
    input wire clk, reset, load,
    input wire [31:0] PCnext,
    output reg [31:0] PC
);
always @(posedge clk or negedge reset) begin
    if (!reset)      PC <= 32'b0;
    else if (load)   PC <= PCnext;
end
endmodule
```

4.2 Instruction Memory (InstrMem.v)

A 64-entry × 32-bit synchronous ROM. The 21-word Fibonacci machine-code program is pre-loaded via \$readmemh at simulation time. Read is asynchronous: the output Instr is available the same cycle the PC is applied. Word-alignment is enforced by using A[7:2] as the array index, discarding the always-zero bottom two bits of any valid instruction address.

```
module InstrMemory (
    input  wire [31:0] A,
    output wire [31:0] RD
);
    reg [31:0] InstrMem [0:63];
    initial $readmemh("program.hex", InstrMem);
    assign RD = InstrMem[A[7:2]];
endmodule
```

4.3 Register File (Register_file.v)

Implements the 32 general-purpose RISC-V registers (x0–x31), each 32 bits wide. Two asynchronous read ports (RD1, RD2) allow simultaneous operand fetch in the same cycle as decode. One synchronous write port (WD3) updates on the rising clock edge when RegWrite is asserted and the destination is not x0. Register x0 is permanently hardwired to zero on both read ports, satisfying the RISC-V ISA requirement.

4.4 Sign Extend (SignExtend.v)

Reconstructs the 32-bit sign-extended immediate from whichever bit fields the current instruction format uses. The ImmSrc control signal (2 bits, from the Main Decoder) selects one of three reconstruction patterns, matching the three immediate formats present in the instruction set:

ImmSrc	Format	Bit Reconstruction	Used By
00	I-type	{20{Instr[31]}}, Instr[31:20]	addi, slli, lw
01	S-type	{20{Instr[31]}}, Instr[31:25], Instr[11:7]	sw
10	B-type	{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0	beq, bne, blt

The B-type pattern is the most complex: branch offsets are always even (bit 0 is implicitly 0), and the remaining bits are scattered across the instruction word to keep other fields in fixed positions across all formats.

4.5 Generic 2:1 MUX (MUX21.v)

A parameterised 32-bit 2:1 multiplexer used three times in the datapath: selecting the ALU's second operand (register vs. immediate), selecting the write-back value (ALU result vs. memory read data), and selecting the next PC (PC+4 vs. branch target). Factoring this into a single reusable module avoids duplicating the select logic and makes the top-level connections self-documenting.

4.6 Arithmetic Logic Unit (ALU.v)

A combinational block implementing seven operations selected by the 3-bit ALUControl signal from the ALU Decoder. Two status flags are derived from ALUResult: Zero (all bits zero, used by BEQ/BNE) and Sign (bit 31 set, used by BLT as a negative-result indicator). The default case outputs zero to prevent undefined behaviour for unused encodings.

ALUControl	Operation	Used by
000	$A + B$	add, addi, lw/sw address calc
001	$A \ll B[4:0]$	sll, slli
010	$A - B$	sub, all branch comparisons
100	$A \wedge B$	xor
101	$A \gg B[4:0]$	srl, srli
110	$A \mid B$	or
111	$A \& B$	and

Shift amounts are masked to B[4:0] (5 bits): a 32-bit number only has 32 bit positions (0–31), so any shift amount above 31 is physically meaningless and produces the same result as (amount mod 32). Masking prevents unintended behaviour if a large immediate is ever supplied as SrcB.

Right shifts are implemented as logical ($\gg$), not arithmetic ($\ggg$). This is a deliberate design choice documented here: the specification does not specify the shift direction, and the Fibonacci test program uses only left shifts (slli), so this assumption has no impact on verification results.

4.7 Data Memory (DataMemory.v)

A 64-entry $\times$ 32-bit RAM with asynchronous read and synchronous write. Only store (SW) instructions write to data memory; the write-enable port WE is driven directly by the MemWrite signal from the Main Decoder,

ensuring the memory is inert on every non-store cycle. The address supplied by the ALU (ALUResult, the computed base+offset) is word-aligned via the same A[7:2] indexing used in the Instruction Memory.

4.8 Main Decoder (MainDecoder.v)

A combinational lookup table mapping the 7-bit opcode to all high-level datapath control signals. It is the primary switch that determines the 'shape' of a cycle: whether a register is written, which ALU operand is selected, whether memory is accessed, and whether this instruction might redirect the PC. The ALUOp output is a 2-bit hint passed to the ALU Decoder rather than a complete 3-bit operation code, because the opcode alone is not sufficient to determine the exact ALU operation for R-type and I-type instructions.

Instruction	Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	0	0	00
R-type	0110011	1	00	0	0	0	0	10
I-type	0010011	1	00	1	0	0	0	10
branch	1100011	0	10	0	0	0	1	01
default	-----	0	10	0	0	0	0	00

4.9 ALU Decoder (AluDecoder.v)

A second combinational lookup table that refines ALUOp into the 3-bit ALUControl signal by additionally inspecting funct3 and two bits of the instruction word: op5 (Instr[5], which distinguishes R-type opcode 0110011 from I-type opcode 0010011) and funct7b5 (Instr[30], which is the only bit distinguishing ADD from SUB — both share funct3=000 and the same R-type opcode).

When ALUOp is 00 or 01, the ALU operation is fully determined by the instruction class (always add for lw/sw, always subtract for branches), and funct3/funct7 are ignored. Only when ALUOp is 10 (R-type or I-type arithmetic) does the nested case statement inspect funct3 to select the specific operation.

4.10 Branch Resolution Logic (BranchResolution.v)

Determines whether a branch instruction is actually taken, given the ALU's comparison result. The ALU Decoder always routes branch instructions through subtraction (ALUControl=010); this module then interprets the resulting flags according to which branch type was encountered, identified by funct3:

- BEQ (funct3=000): branch taken when Zero=1 (operands were equal).
- BNE (funct3=001): branch taken when Zero=0 (operands were not equal).
- BLT (funct3=100): branch taken when Sign=1 (subtraction produced a negative result, i.e., $rs1 < rs2$ in two's complement).

The Branch control signal from the Main Decoder acts as a gate: if Branch=0 (the instruction is not a branch type), PCSrc is forced to 0 regardless of the flag values, preventing spurious jumps.

4.11 HALT Detection

RV32I has no dedicated halt instruction. The Fibonacci program terminates with the word 0x00000000 (an all-zero instruction whose opcode 00000000 is not a legal RISC-V encoding). This is treated as the sentinel:

```
assign load = (Instr != 32'b0);
```

When the PC reaches the halt word, load is driven low. The PC register's always block executes but the else-if branch is not taken (load=0), leaving PC unchanged. The processor re-fetches the halt sentinel every clock cycle thereafter, indefinitely, without advancing or modifying any state.

4.12 Top-Level Module (TopRiscV.v / riscv_top)

Instantiates all eleven blocks and connects them with the internal wires implied by the datapath diagram. The module has only two external ports — clk and reset — making it completely self-contained for simulation purposes. Register-file addresses are wired to fixed bit slices of the instruction word (rs1 = Instr[19:15], rs2 = Instr[24:20], rd = Instr[11:7]) without any additional decode logic, exploiting RISC-V's fixed-field encoding.

5. Simulation and Verification

5.1 Test Program

The processor was verified by executing the 21-word Fibonacci program supplied by IEEE CUSB as a hexadecimal machine-code file. The program initializes eight registers, then executes a 10-iteration loop that computes successive Fibonacci numbers and stores each result in data memory using an alternating-register scheme controlled by a selector register (x3). The loop uses all five supported instruction types: R-type arithmetic, I-type immediate, load/store, and all three branch variants.

5.2 Program Execution Summary

Address	Instruction	Operation
0x00	xor x0,x0,x0	Ensure x0=0
0x04	addi x1,x0,0	x1 (fib_a) $\leftarrow$ 0
0x08	addi x2,x0,1	x2 (fib_b) $\leftarrow$ 1
0x0C	addi x3,x0,1	x3 (selector) $\leftarrow$ 1
0x10	addi x4,x0,1	x4 (constant 1) $\leftarrow$ 1
0x14	addi x5,x0,0	x5 (loop counter) $\leftarrow$ 0
0x18	addi x6,x0,10	x6 (loop limit) $\leftarrow$ 10
0x1C	addi x7,x0,0	x7 (store address) $\leftarrow$ 0
0x20	beq x3,x4,eq	Loop: if selector=1, jump to eq path
0x24	add x2,x2,x1	Not-eq path: x2 $\leftarrow$ x2+x1
0x28	sub x3,x3,x4	selector $\leftarrow$ 1
0x2C	slli x7,x5,2	x7 $\leftarrow$ x5 $\times$ 4
0x30	sw x2,0(x7)	mem[x7] $\leftarrow$ x2
0x34	beq x4,x4,+20	Unconditional jump to counter increment
0x38	add x1,x1,x2	Eq path: x1 $\leftarrow$ x1+x2
0x3C	add x3,x3,x4	selector $\leftarrow$ 2
0x40	slli x7,x5,2	x7 $\leftarrow$ x5 $\times$ 4
0x44	sw x1,0(x7)	mem[x7] $\leftarrow$ x1
0x48	addi x5,x5,1	x5 $\leftarrow$ x5+1
0x4C	blt x5,x6,-44	if x5<10, jump back to loop
0x50	0x00000000	HALT — PC freezes

5.3 Expected Results

After executing 83 clock cycles (8 setup + 10 loop passes $\times$ 7–8 instructions each + halt fetch), all monitored signals must match the following ground-truth values, derived by both manual instruction-level trace and Python behavioural simulation cross-checked against the C reference model in the project specification:

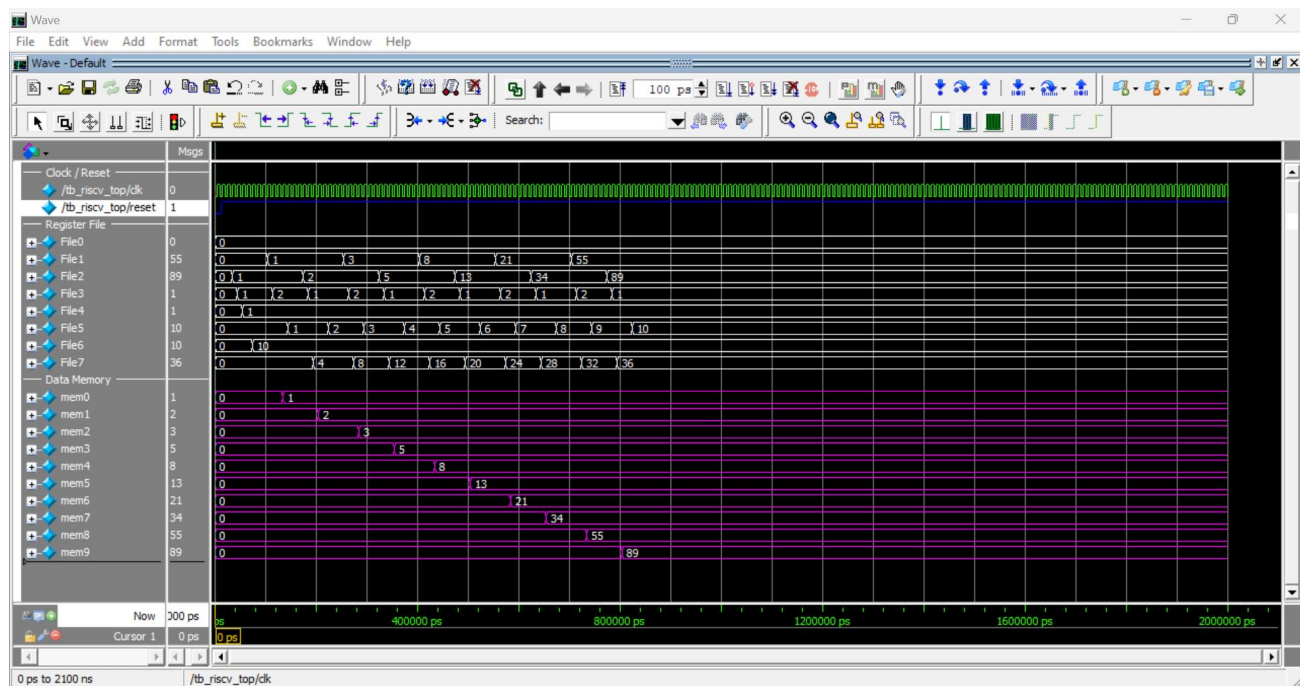
Signal	Final Value	Meaning
File[0] (x0)	0	Hardwired zero — constant throughout
File[1] (x1)	55	Last Fibonacci value written on the equal path
File[2] (x2)	89	Last Fibonacci value written on the not-equal path
File[3] (x3)	1	Selector register, ends at 1 after 10 iterations
File[4] (x4)	1	Constant 1 — never modified
File[5] (x5)	10	Loop counter, exits when it equals x6
File[6] (x6)	10	Loop limit — never modified
File[7] (x7)	36	Last store address = 9 $\times$ 4

Memory Location	mem[] Index	Value	Fibonacci Term
Address 0x00	mem[0]	1	F(1)
Address 0x04	mem[1]	2	F(2) — 0+1+1=2
Address 0x08	mem[2]	3	F(3)
Address 0x0C	mem[3]	5	F(4)
Address 0x10	mem[4]	8	F(5)
Address 0x14	mem[5]	13	F(6)
Address 0x18	mem[6]	21	F(7)
Address 0x1C	mem[7]	34	F(8)
Address 0x20	mem[8]	55	F(9)
Address 0x24	mem[9]	89	F(10)

5.4 Simulation Environment

Simulation was performed in ModelSim using Verilog-2001. A ModelSim automation script (run.do) compiles all source files, elaborates the testbench, configures the waveform window to display File[0]–File[7] and mem[0]–mem[9] in unsigned decimal format (matching the reference waveform's radix), and runs the simulation for 2000 ns — approximately 2.2× the minimum time needed to reach HALT at ~920 ns.

The testbench drives an active-low asynchronous reset (held low for 12 ns to guarantee reset assertion across the first clock edge) and then releases it, allowing the processor to begin fetching from address 0. No external stimulus is needed beyond reset and clock: the program is self-contained inside the instruction memory.



6. Design Decisions and Assumptions

- Shift direction: right shifts are implemented as logical (>>). The specification does not specify logical vs. arithmetic; this choice is documented explicitly and has no impact on the verification program.
- HALT detection: the all-zero word (0x00000000) at program end is used as a halt sentinel. The load signal to the PC register is asserted inline in the top module as (Instr != 32'b0). This is not a separate synthesisable block; it is a one-line combinational assignment.

- x0 hardwiring: x0 is forced to zero on both read ports of the register file regardless of what is written to it, satisfying the RISC-V ISA permanent-zero constraint. The write guard (A3 != 0) prevents the register array from being driven unnecessarily.
- Don't-care outputs: the Main Decoder's truth table contains X entries for ResultSrc when RegWrite=0 (sw, branches) and for ImmSrc on R-type instructions. These are replaced with concrete zero values in the implementation to prevent simulation red-lines (X propagation) in paths that are functionally unused on those cycles.
- Memory initialisation: both instruction memory (via \$readmemh) and data memory (via initial block) are pre-zeroed at simulation time zero. The async-reset path additionally zeroes the register file on reset assertion, so there is no dependence on implicit Verilog initial-value semantics for functional correctness.
- Word alignment: both memories use A[7:2] as the array index. This discards the always-zero bottom two bits of any word-aligned address and maps byte addresses 0, 4, 8, ... to array indices 0, 1, 2, ..., correctly spanning the 64-entry arrays with 6-bit indices.

7. Skills Demonstrated

Skill	Application in this Project
RTL Verilog design	12 synthesisable modules written from specification, covering combinational and sequential logic, parameterised modules, and hierarchical instantiation
Computer architecture	Harvard single-cycle microarchitecture; instruction decoding; datapath vs. control separation; program counter, register file, ALU, and memory subsystems
Instruction set architecture	RV32I R/I/S/B-type encoding; sign extension for each format; funct3/funct7 sub-opcode decoding; branch offset reconstruction
RTL debugging	Identified and resolved four classes of bug: sensitivity-list errors (sync vs. async reset), combinational-vs-sequential memory write errors, MUX port-ordering errors, and HALT logic removal
HDL simulation	ModelSim/QuestaSim Tcl automation; waveform configuration; radix selection; simulation time budgeting
Systematic verification	Manual instruction-level trace; cross-check against specification reference waveform and C reference model

8. Tools and Environment

- Language: Verilog-2001 (IEEE Std 1364-2001)
- Simulator: ModelSim DE (Mentor Graphics / Siemens EDA)
- Development machine: HP ProBook 640 G8, Intel Core i5-1135G7, 16 GB RAM.
- Version control: GitHub (github.com/Poulakerolos)
- Reference: RISC-V Unprivileged ISA Specification v20191213; Harris & Harris — Digital Design and Computer Architecture RISC-V Edition

9. Conclusion

A fully functional 32-bit single-cycle RISC-V processor was designed, implemented, debugged, and verified in RTL Verilog within nine days as a competitive final project for the IEEE Cairo University Digital Design Workshop. The processor correctly executes a 21-instruction Fibonacci machine-code program, producing the ten expected Fibonacci values (1, 2, 3, 5, 8, 13, 21, 34, 55, 89) in data memory and the correct final register-file state, matching the specification's reference waveform exactly.

The project required understanding processor microarchitecture from first principles, translating a block-diagram specification into twelve synthesisable Verilog modules, and systematically identifying and resolving four classes of RTL bug across the design and testbench. The result is a clean, well-structured baseline single-cycle core ready for extension to a pipelined implementation.